

AIAC

Lab Assignment - 20.5

Name: G.Anoop Goud

HT. No: 2303A51085

Batch: 02

Task 1 – Password Storage Vulnerability Check

Task: Analyze an AI-generated signup script for insecure password storage.

Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

- A secure registration script with hashed password storage.

Copilot Suggested Code :

```
task1.py
1 # Task: Analyze an AI-generated signup script for insecure password storage.
2
3 # AI-generated user registration program
4 def register_user(username, password):
5     # Insecure password storage (plain text)
6     with open('users.txt', 'a') as file:
7         file.write(f'{username}:{password}\n')
8     print("User registered successfully!")
9     # Example usage
10    register_user('john_doe', 'password123')
11
12    # Improved registration script with hashed password storage
13    import bcrypt
14    def register_user_secure(username, password):
15        # Hash the password using bcrypt
16        hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
17        with open('users_secure.txt', 'a') as file:
18            file.write(f'{username}:{hashed_password.decode("utf-8")}\n')
19        print("User registered securely!")
20    # Example usage
21    register_user_secure('john_doe', 'password123')
22
23    # Analysis:
24    # The original registration script stores passwords in plain text, which is a significant security risk. If the 'users.txt' file is compromised, attackers c
25    # The improved registration script uses bcrypt to hash passwords before storing them. This means that even if the 'users_secure.txt' file is compromised, at
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_1.py
● User registered successfully!
● User registered securely!
```

Task 2 – Hardcoded Secrets Detection

Task: Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

Copilot Suggested Code :

```
task_2.py > ...
1  # Task: Evaluate AI-generated code for hardcoded API keys or credentials.
2
3  # AI-generated script with hardcoded API key
4  import requests
5  import os
6
7  def fetch_data_secure():
8      api_key = os.getenv('API_KEY')
9
10     if not api_key:
11         print("API key not found.")
12         return
13
14     url = f'https://jsonplaceholder.typicode.com/posts'
15
16     try:
17         response = requests.get(url, timeout=5)
18         response.raise_for_status()
19         print("Data fetched successfully!")
20         print(response.json()[0:2]) # sample output
21
22     except requests.exceptions.RequestException as e:
23         print("Error:", e)
24
25     fetch_data_secure()
26
27 # Analysis:
28 # The original script contains a hardcoded API key, which is a security risk. If the code is shared or exposed, the API key can be easily extracted and misu
29 # The refactored script retrieves the API key from an environment variable, which is a more secure practice. This way, the API key is not exposed in the cod
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/Lab_assignment-20.5/task_2.py
API key not found.
```

Task 3 – File Upload Security Validation

Task: Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename Handling.

Expected Output:

- A secure file upload implementation.

Copilot Suggested Code :

```
task_3.py > ...
1 # Task: Test an AI-generated file upload program for unsafe file handling.
2
3 # AI-generated Flask file upload example with vulnerabilities
4 from flask import Flask, request, redirect, url_for
5 from werkzeug.utils import secure_filename
6 import os
7 app = Flask(__name__)
8 UPLOAD_FOLDER = 'uploads'
9 app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
10 ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}
11
12 def allowed_file(filename):
13     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
14
15 @app.route('/upload', methods=['GET', 'POST'])
16 def upload_file():
17     if request.method == 'POST':
18         if 'file' not in request.files:
19             return "No file part"
20         file = request.files['file']
21         if file.filename == '':
22             return "No selected file"
23         if file and allowed_file(file.filename):
24             filename = secure_filename(file.filename)
25             file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
26             return "File uploaded successfully"
27         return ""
28     <!doctype html>
29     <title>Upload new File</title>
30     <h1>Upload new File</h1>
31     <form method=post enctype=multipart/form-data>
32         <input type=file name=file>
33         <input type=submit value=Upload>
34     </form>
35     ...
36 if __name__ == '__main__':
37     if not os.path.exists(UPLOAD_FOLDER):
38         os.makedirs(UPLOAD_FOLDER)
39     app.run(debug=True)
40
41 # Analysis:
42 # The original AI-generated file upload program has several vulnerabilities. It does not validate the file type or size, which can lead to the upload of mal
43 # The refactored implementation includes a function to check for allowed file types and uses the secure_filename function to ensure that the uploaded file's
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_3.py
* Serving Flask app 'task_3'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 854-020-678
```

Task 4 – Command Injection Prevention

Task: Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

Copilot Suggested Code :

```
task_4.py > ...
1 # Task: Analyze an AI-generated Python script that executes system commands.
2
3 # AI-generated code with command injection vulnerability
4 import os
5 def execute_command(user_input):
6     command = f"echo {user_input}"
7     os.system(command)
8 # Example of vulnerable usage
9 user_input = input("Enter a string to echo: ")
10 execute_command(user_input)
11 # Analysis:
12 # The original AI-generated code is vulnerable to command injection because it directly incorporates user input into a system command without any validation
13 # To fix this vulnerability, we can use the subprocess module with a list of arguments instead of a single string command. This way, the user input is treated as a single argument.
14 import subprocess
15
16 def execute_command_safe(user_input):
17     # Proper validation
18     if not all(c.isalnum() or c.isspace() for c in user_input):
19         raise ValueError("Invalid input")
20
21     # Safe execution
22     subprocess.run(["cmd", "/c", "echo", user_input], check=True)
23
24 user_input = input("Enter a string to echo: ")
25
26 try:
27     execute_command_safe(user_input)
28 except ValueError as e:
29     print(e)
30 # In this refactored code, we use subprocess.run() with a list of arguments, which prevents command injection. Additionally, we validate the user input to ensure it only contains alphanumeric characters and spaces.
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_4.py
Enter a string to echo: hello
hello
Enter a string to echo: mendis
mendis
```

Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation.

Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.

Copilot Suggested Code :

```
task_5.py > ...
1 # Task: Check AI-generated web form code for improper input validation.
2
3 # AI-generated code for a feedback form using Flask
4 from flask import Flask, request, render_template_string
5 app = Flask(__name__)
6 @app.route('/feedback', methods=['GET', 'POST'])
7 def feedback():
8     if request.method == 'POST':
9         user_feedback = request.form['feedback']
10         # Unsafe rendering of user input
11         return render_template_string(f"<h1>Your Feedback: {user_feedback}</h1>")
12     return ''
13     <form method="post">
14         <textarea name="feedback"></textarea>
15         <input type="submit" value="Submit">
16     </form>
17
18 if __name__ == '__main__':
19     app.run(debug=True)
20
21 # Analysis:
22 # The original AI-generated code is vulnerable to Cross-Site Scripting (XSS) attacks because it directly renders user input without any sanitization. An attacker can exploit this vulnerability by injecting malicious code into the user input. To fix this vulnerability, we can use the 'escape' function from the 'markupsafe' module to sanitize the user input before rendering it. This will ensure that any malicious code is escaped and rendered as plain text, preventing it from being executed.
23 from flask import Flask, request, render_template_string
24 from markupsafe import escape
25 app = Flask(__name__)
26 @app.route('/feedback', methods=['GET', 'POST'])
27 def feedback():
28     if request.method == 'POST':
29         user_feedback = request.form['feedback']
30         # Safe rendering of user input
31         safe_feedback = escape(user_feedback)
32         return render_template_string(f"<h1>Your Feedback: {safe_feedback}</h1>")
33     return ''
34     <form method="post">
35         <textarea name="feedback"></textarea>
36         <input type="submit" value="Submit">
37     </form>
38
39 if __name__ == '__main__':
40     app.run(debug=True)
41
42 # In this refactored code, we use the 'escape' function to sanitize the user input before rendering it. This prevents any malicious code from being executed
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-20.5/task_5.py
* Serving Flask app 'task_5'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 854-020-678
* Detected change in 'c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5\task_5.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 854-020-678
* Detected change in 'c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5\task_5.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 854-020-678
* Detected change in 'c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5\task_5.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 854-020-678
* Detected change in 'c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-20.5\task_5.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 854-020-678
```